

# Compiler Design - Module 1

## Top-Down Parsing - Basic Level Study Guide

Recursive Descent • Predictive Parsing • Left Recursion • Left Factoring

### 1. Top-Down Parsing

#### Definition

Top-down parsing builds the parse tree from **root**  $\rightarrow$  **leaves**.

From PDF:

Top down parsing constructs parse tree from start symbol and finds leftmost derivation.

#### Idea

Start symbol

↓

Expand productions

↓

Match input string

#### Example

Grammar:

$S \rightarrow aA$

$A \rightarrow b$

Input = ab

Derivation:

S

$\Rightarrow aA$

$\Rightarrow ab$

Diagram:

S

/ \

a A

|

b

## 2. Recursive Descent Parsing

### Definition

Recursive descent = top-down parser using **functions for each nonterminal**.

From PDF:

It is the most general form of top-down parsing and may involve backtracking.

### Example from PDF

Grammar:

$S \rightarrow cAd$

$A \rightarrow ab \mid a$

Input = cad

Step 1:

$S \rightarrow cAd$

Step 2 - try  $A \rightarrow ab$

c a b d (fails)

Backtrack

Step 3 - try  $A \rightarrow a$

c a d (success)

Diagram:

```
S
/ | \
c A d
|
a
```

From notes page showing same example

### Problem

Recursive descent fails when:

- grammar is left recursive
- grammar is ambiguous

### 3. Predictive Parsing

#### Definition

Predictive parser = recursive descent **without backtracking**.

From PDF:

Predictive parsing chooses production using next input symbol.

Conditions:

Remove left recursion

Left factoring required

#### Example

Grammar:

$$E \rightarrow E + T \mid T$$

This is left recursive.

Convert:

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \epsilon$$

Now predictive parser can work.

#### Predictive Parsing Idea

Look at next input

Choose correct rule

Expand

Repeat

## 4. Left Recursion

### Definition

Grammar is left recursive if:

$$A \rightarrow A \alpha$$

From PDF:

Left recursion causes infinite loop in top-down parser.

### Example

$$E \rightarrow E + T \mid T$$

If parser tries first rule:

$$E \rightarrow E + T$$

$$\rightarrow E + T + T$$

$$\rightarrow \text{infinite}$$

### Remove Left Recursion

Rule:

$$A \rightarrow A \alpha \mid \beta$$

Convert to:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

### Example

$$E \rightarrow E + T \mid T$$

Becomes:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

From notes page

## 5. Left Factoring

### Definition

Used when grammar has **common prefix**.

From PDF:

Used when parser cannot decide which production to use.

### Example

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

Factor:

$$A \rightarrow \alpha B$$
$$B \rightarrow \beta_1 \mid \beta_2$$

### Example (dangling else)

Original:

$$\begin{aligned} \text{stmt} &\rightarrow \text{if cond then stmt else stmt} \\ &\mid \text{if cond then stmt} \end{aligned}$$

Problem  $\rightarrow$  both start with same part

Left factoring:

$$\begin{aligned} \text{stmt} &\rightarrow \text{if cond then stmt else\_part} \\ \text{else\_part} &\rightarrow \text{else stmt} \mid \epsilon \end{aligned}$$

From PDF

## 6. Non-Recursive Predictive Parser

### Definition

Predictive parser using:

- stack
- parsing table
- input buffer

From PDF:

Nonrecursive parser uses stack and parsing table.

### Components

1. Stack
2. Input buffer
3. Parsing table
4. Driver

Diagram (from PDF):

```
input → a + b $  
↓  
Predictive parser  
↑ ↓  
stack parsing table
```

From figure in notes

### Working

Case 1:

```
Top = terminal and matches input  
→ pop  
→ move input
```

Case 2:

```
Top = nonterminal  
→ use table  
→ replace by production
```

Case 3:

```
Top = $ and input = $  
→ accept
```

### Example

Stack:

\$ S

Input:

a b \$

Use table → expand → match → accept

## Summary Table

| Topic                | Meaning                 |
|----------------------|-------------------------|
| Top-down             | build tree from root    |
| Recursive descent    | uses functions          |
| Predictive parser    | no backtracking         |
| Left recursion       | $A \rightarrow A\alpha$ |
| Left factoring       | remove common prefix    |
| Non-recursive parser | stack + table           |

---

This is a basic-level study guide for Top-Down Parsing in Module 1 - Compiler Design. Topics include recursive descent parsing, predictive parsing, left recursion removal, left factoring, and non-recursive parsing techniques.